

Spring Framework — Day 23 Notes

Annotations (Day 06): @Value, Properties Files & @PropertySource

1. Making @Component Visible to the IoC Container

For @Component to become visible to the IoC container — so that it can load the class, create the object, perform dependency injection, and manage its life cycle — should we simply add <context:annotation-config> in the XML file?

No. <context:annotation-config> only enables the BeanPostProcessors needed for annotations that are already present inside a class. It cannot read stereotype annotations.

That is why we need:

```
<context:component-scan base-package="" />
```

This tag can read all the stereotype annotations, so the IoC container can perform its operations. But for the other annotations present inside the class, we still need <context:annotation-config>, which is internally imported by <context:component-scan>. So we don't have to write both tags separately.

Note

Since we are moving away from XML altogether, it is better to use the type of IoC container that can read all the annotations present inside a class and call all the BeanPostProcessors needed to process them. That is why we use:

```
AnnotationConfigApplicationContext();
```

2. The Problem: Assigning Values Without XML

To perform some operations, we need values assigned to variables. Previously, this was done through XML, using setter or constructor injection. But now that we have stopped using XML, how do we assign values?

For this, we use a properties file along with the @Value annotation. Let's first check how this works on fields.

Employee.java — Field Injection with @Value

```

package com.day23;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component("ForAllEmployee")
@Scope("prototype")
public class Employee {
    @Value("prashant")
    private String employeeName;
    @Value("NIT0005")
    private String employeeId;
    @Value("8")
    private int employeeExp;
    @Value("true")
    private boolean allProjectStatus;
    @Value("50000")
    private long employeeBasicSalary;

    public long getIncrementSalary() {
        if (allProjectStatus)
            employeeBasicSalary = employeeBasicSalary + 100000;
        return employeeBasicSalary;
    }

    @Override
    public String toString() {
        return "Employee [employeeName=" + employeeName + ", employeeId="
+ employeeId
        + ", employeeExp=" + employeeExp + ", allProjectStatus=" +
allProjectStatus
        + ", employeeBasicSalary=" + employeeBasicSalary + "];"
    }
}

```

EmployeePortal.java — Main Class

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationC
ontext;

public class EmployeePortal {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext("com.day23.backup");

        Employee bean1 = context.getBean("ForAllEmployee", Employee.class)
;

        System.out.println(bean1.getIncrementSalary());
    }
}

```

3. Can We Do the Same Using Setters?

Yes. Example:

```

package com.day23.backup;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component("ForAllEmployee")
@Scope("prototype")
public class Employee {

    private String employeeName;
    private String employeeId;
    private int employeeExp;
    private boolean allProjectStatus;
    private long employeeBasicSalary;

    @Value("prashant")
    public void setEmployeeName(String employeeName) {
        this.employeeName = employeeName;
    }

    @Value("NIT0005")
    public void setEmployeeId(String employeeId) {
        this.employeeId = employeeId;
    }

    @Value("8")
    public void setEmployeeExp(int employeeExp) {
        this.employeeExp = employeeExp;
    }
}

```

```

        @Value("true")
        public void setAllProjectStatus(boolean allProjectStatus) {
            this.allProjectStatus = allProjectStatus;
        }

        @Value("50000")
        public void setEmployeeBasicSalary(long employeeBasicSalary) {
            this.employeeBasicSalary = employeeBasicSalary;
        }

        public long getIncrementSalary() {
            if (allProjectStatus)
                employeeBasicSalary = employeeBasicSalary + 100000;
            return employeeBasicSalary;
        }

        @Override
        public String toString() {
            return "Employee [employeeName=" + employeeName + ", employeeId="
+ employeeId
                + ", employeeExp=" + employeeExp + ", allProjectStatus=" +
allProjectStatus
                + ", employeeBasicSalary=" + employeeBasicSalary + "]";
        }
    }
}

```

With the same main class, we will get the same output. But the issue here is that we are hard-coding the values.

4. Solving Hard-Coded Values with a Properties File

We can resolve the hard-coding problem by using a properties file. We create the properties file with any name, but the recommended name is **application.properties**.

application.properties

```

Employee_name=Prashant
Employee_id=NIT0005
Employee_exp=7
Employee_projectStatus=true
Employee_basicSalary=50000

```

We have to tell the IoC container about this properties collection. We do this with the help of the configuration file.

application-context.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans
    xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:util="http://www.springframework.org/schema/util"

    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd
        http://www.springframework.org/schema/util
        https://www.springframework.org/schema/util/spring-util.xsd">

    <util:properties id="myprops"
        location="classpath:com/day23/backup/application.properties">
    </util:properties>

    <!-- Since we are using ClassPathXmlApplicationContext, we still need
this
        tag below, because the bean is not declared inside the configurat
ion
        file - instead, it is declared using @Component. -->
    <context:component-scan base-package="com.day23.backup">
    </context:component-scan>

</beans>

```

Now, how do we read the value from that properties file inside the class that is managed by the IoC container?

Employee.java — Reading Values with #{ }

```

@Component("ForAllEmployee")
@Scope("prototype")
public class Employee {

    @Value("#{myprops.Employee_name}")
    private String employeeName;
    @Value("#{myprops.Employee_id}")
    private String employeeId;
    @Value("#{myprops.Employee_exp}")
    private int employeeExp;
    @Value("#{myprops.Employee_projectStatus}")
    private boolean allProjectStatus;
    @Value("#{myprops.Employee_basicSalary}")
    private long employeeBasicSalary;

    public long getIncrementSalary() {
        if (allProjectStatus)
            employeeBasicSalary = employeeBasicSalary + 100000;
        return employeeBasicSalary;
    }

    @Override
    public String toString() {
        return "Employee [employeeName=" + employeeName + ", employeeId="
+ employeeId
        + ", employeeExp=" + employeeExp + ", allProjectStatus=" +
allProjectStatus
        + ", employeeBasicSalary=" + employeeBasicSalary + "];"
    }
}

```

EmployeePortal.java — Main Class

```

public class EmployeePortal {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext(
            "com/day23/backup/application-context.xml");
        // AnnotationConfigApplicationContext("com.day23.backup");

        Employee bean1 = context.getBean("ForAllEmployee", Employee.class)
;

        System.out.println(bean1.getIncrementSalary());

        System.out.println(bean1);
    }
}

```

Running this will give the exact output based on the properties file values.

5. Switching to a Different Properties File

Suppose we want some different data. Instead of changing the same properties file, let's create a new one, so that we have information about each and every employee.

application1.properties

```
Employee_name=Mahi  
Employee_id=NIT0007  
Employee_exp=7  
Employee_projectStatus=true  
Employee_basicSalary=70000
```

Inside the XML file, we just change the classpath location:

Before

```
<util:properties id="myprops"  
    location="classpath:com/day23/backup/application.properties">  
</util:properties>
```

After

```
<util:properties id="myprops"  
    location="classpath:com/day23/backup/application1.properties">  
</util:properties>
```

Run the same main method class and try this to get the output of your choice.

6. Doing the Same Without XML — @PropertySource

Now that we are not using XML, is it possible to read a properties file directly through annotations? Yes — using **@PropertySource**.

Employee.java — Using @PropertySource and \${ }

```

@Component("ForAllEmployee")
@Scope("prototype")
@PropertySource("classpath:com/day23/backup/application1.properties")
public class Employee {

    @Value("${Employee_name}")
    private String employeeName;
    @Value("${Employee_id}")
    private String employeeId;
    @Value("${Employee_exp}")
    private int employeeExp;
    @Value("${Employee_projectStatus}")
    private boolean allProjectStatus;
    @Value("${Employee_basicSalary}")
    private long employeeBasicSalary;

    public long getIncrementSalary() {
        if (allProjectStatus)
            employeeBasicSalary = employeeBasicSalary + 100000;
        return employeeBasicSalary;
    }

    @Override
    public String toString() {
        return "Employee [employeeName=" + employeeName + ", employeeId="
+ employeeId
        + ", employeeExp=" + employeeExp + ", allProjectStatus=" +
allProjectStatus
        + ", employeeBasicSalary=" + employeeBasicSalary + "]";
    }
}

```

Using the same application.properties and application1.properties files, run the main method by changing ClassPathXmlApplicationContext to AnnotationConfigApplicationContext:

```

public class EmployeePortal {
    public static void main(String[] args) {
        ApplicationContext context =
            new AnnotationConfigApplicationContext("com.day23.backup");

        Employee bean1 = context.getBean("ForAllEmployee", Employee.class)
;

        System.out.println(bean1.getIncrementSalary());

        System.out.println(bean1);
    }
}

```

Try this and check the output.

If you want to switch from one properties file to another, just change the path inside @PropertySource:

Before

```
@PropertySource("classpath:com/day23/backup/application.properties")
```

After

```
@PropertySource("classpath:com/day23/backup/application1.properties")
```

7. How @PropertySource Actually Works

With `<util:properties>` in XML, Spring reads the properties file and creates a separate bean (like **myprops**) that holds the file's contents as a collection. That is why we access values through that bean's name, using `#{myprops.propertyName}`.

`@PropertySource` does not work the same way. It does not create a separate properties bean-like collection. Instead, it reads the properties file and dumps its content directly into the IoC container's environment. Because the values become part of the container's environment itself, `@Value` can pick them up directly using `${propertyName}`, without needing to refer to any bean name.

8. Common Mistakes to Avoid

Mistake 1 — Missing Schema for util:properties

Watch Out

If you are using `application-context.xml` to read a properties file with `<util:properties>`, you must add the proper util schema (`xmlns:util` and its `schemaLocation`) in the XML file. Without this schema, `<util:properties>` will not work.

Mistake 2 — Wrong File Naming

Watch Out

While creating the properties file, do not name it as `application-properties`. Since we are creating the configuration file as `application-context.xml`, a wrongly named properties file (like `application-properties` instead of `application.properties`) means the IoC container will not be able to read it.

Mistake 3 — Wrong Syntax for Reading Values

Watch Out

If you are using `application-context.xml` to read the properties file content inside your class, use `#{idname.propertyFileName}` — for example, `#{myprops.Employee_name}`. But if you are not using an XML configuration file, then you can directly write `${propertyFileName}` — for example, `#{Employee_name}`.